Sinusoidal Positional Encoding in Transformers

A Detailed Beginner-Friendly Mini-Book

Shahbod Sobhkhiz

Chapter 1 Why Transformers Need Positional Encoding

The Core Problem

A Transformer does not naturally know the order of tokens.

Unlike a recurrent neural network, which reads tokens one by one, or a convolutional neural network, which has some built-in local neighborhood structure, a vanilla Transformer receives all tokens at the same time. Self-attention compares every token with every other token, but self-attention by itself does not contain a built-in idea of first token, second token, previous token, or next token.

For example, consider:

```
I love cats
cats love I
```

The words are the same, but the order is different. To humans, the difference is obvious. To a pure self-attention layer without position information, the model mostly sees the same token vectors arranged in different rows. The architecture can process the rows, but it has no explicit signal saying which row means "first", which row means "second", and so on.

So we need to inject position information into the model. That is the job of **positional encoding**.

Token Meaning Versus Token Position

Each token embedding represents what the token is. For example:

```
cat -> vector representing the meaning of cat
sat -> vector representing the meaning of sat
mat -> vector representing the meaning of mat
```

But the model also needs to know where each token is. So for each position, we create a position vector:

```
position 0 -> positional vector
position 1 -> positional vector
position 2 -> positional vector
...
```

In the original Transformer, the token embedding and positional encoding are combined by addition:

$$\text{input vector} = \text{token embedding} + \text{positional encoding}.$$

For the sentence:

```
The cat sat
```

the Transformer receives conceptually:

```
embedding("The") + PE(position 0)
embedding("cat") + PE(position 1)
embedding("sat") + PE(position 2)
```

Now each input vector contains both meaning and position.

Why Not Just Use Raw Numbers?

A beginner's idea might be:

```
position 0 -> 0
position 1 -> 1
position 2 -> 2
position 3 -> 3
```

This is not ideal. First, token embeddings are high-dimensional vectors, often 512-dimensional or larger. A single scalar position number does not naturally match that space. Second, raw numbers grow without bound, so position 10000 is much larger than position 10 and may create scaling problems. Third, the model needs relative structure: position 53 is 3 steps after position 50, just as position 3 is 3 steps after position 0. A good encoding should make this kind of relationship learnable.

Sinusoidal positional encoding was designed to give each position a high-dimensional, bounded, smooth, structured vector.

Chapter 2 The Sinusoidal Positional Encoding Formula

The Formula

The original Transformer uses sine and cosine functions at different frequencies:

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

$$PE(pos, 2i+1) = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right).$$

Even-numbered dimensions use sine. Odd-numbered dimensions use cosine.

Meaning of the Variables

- pos is the token position in the sequence: $0, 1, 2, 3, \ldots$.
- d_{model} is the embedding dimension of the Transformer.
- i is the sine/cosine **pair index**, not exactly the raw dimension index.

If $d_{model} = 512$, every token embedding has 512 numbers, and the positional encoding must also have 512 numbers so that we can add them.

What Exactly Is i?

This was one of the main clarification questions. In the formula, i is not simply "the dimension number." It is the index of a sine/cosine pair.

For example, if $d_{model} = 6$, the six dimensions are grouped like this:

```
dimensions 0 and 1 -> i = 0
dimensions 2 and 3 -> i = 1
dimensions 4 and 5 -> i = 2
```

So:

```
PE(pos, 0) = sin(pos / 10000^(0/6))    because 2i = 0, i = 0
PE(pos, 1) = cos(pos / 10000^(0/6))    because 2i+1 = 1, i = 0

PE(pos, 2) = sin(pos / 10000^(2/6))    because 2i = 2, i = 1
PE(pos, 3) = cos(pos / 10000^(2/6))    because 2i+1 = 3, i = 1

PE(pos, 4) = sin(pos / 10000^(4/6))    because 2i = 4, i = 2
PE(pos, 5) = cos(pos / 10000^(4/6))    because 2i+1 = 5, i = 2
```

Therefore, with $d_{model} = 6$, there are six positional values but only three sine/cosine frequency pairs.

Chapter 3 A Complete Numerical Example

Suppose a token embedding is:

$$[2, 2, 4, 1, 4, 5]$$

This vector has six values, so $d_{model} = 6$. Suppose the token is at position $pos = 2$.

First Pair: Dimensions 0 and 1

For $i = 0$:

$$10000^{2i/d_{model}} = 10000^{0/6} = 1.$$

So:

$$PE(2, 0) = \sin(2) \approx 0.909, \qquad PE(2, 1) = \cos(2) \approx -0.416.$$

Second Pair: Dimensions 2 and 3

For $i = 1$:

$$10000^{2/6} = 10000^{1/3} \approx 21.544.$$

So:

$$PE(2, 2) = \sin(2/21.544) \approx 0.0927, \qquad PE(2, 3) = \cos(2/21.544) \approx 0.9957.$$

Third Pair: Dimensions 4 and 5

For $i = 2$:

$$10000^{4/6} = 10000^{2/3} \approx 464.159.$$

So:

$$PE(2, 4) = \sin(2/464.159) \approx 0.00431, \qquad PE(2, 5) = \cos(2/464.159) \approx 0.99999.$$

Therefore:

$$PE(2) \approx [0.909, -0.416, 0.0927, 0.9957, 0.00431, 0.99999].$$

Adding this to the token embedding gives:

$$\begin{aligned} &[2, 2, 4, 1, 4, 5] + [0.909, -0.416, 0.0927, 0.9957, 0.00431, 0.99999] \\ &\approx [2.909, 1.584, 4.0927, 1.9957, 4.00431, 5.99999]. \end{aligned}$$

This final vector is what enters the first Transformer layer for that token.

Chapter 4 Why Sine and Cosine?

Bounded Values

Sine and cosine always stay between -1 and 1:

$$-1 \leq \sin(x) \leq 1, \qquad -1 \leq \cos(x) \leq 1.$$

This prevents positional values from exploding as positions become large.

Smoothness

Sine and cosine change smoothly. Position 10 and position 11 get different encodings, but they are not completely unrelated. This is useful because nearby words often have related syntactic or semantic roles.

The Sine/Cosine Pair

For one frequency, the pair is:

$$[\sin(pos/s), \cos(pos/s)],$$

where s is the scale. As pos changes, this 2D point moves around a circle. This circular and rotational structure is what makes relative shifts mathematically clean.

Chapter 5 The Relative-Position Property

The Main Clarification

The real distance between tokens does not depend on the absolute position. For example:

$$53 - 50 = 3, \qquad 3 - 0 = 3.$$

The actual distance is the same. The problem is different: a positional encoding function may make the same distance look different depending on where it happens.

A Bad Encoding: $PE(pos) = pos^2$

Suppose:

$$PE(pos) = pos^2.$$

Then two pairs with the same real distance produce very different encoded differences:

$$\begin{aligned} PE(3) - PE(0) &= 3^2 - 0^2 = 9, \\ PE(53) - PE(50) &= 53^2 - 50^2 = 2809 - 2500 = 309. \end{aligned}$$

The distance itself is still 3. But the representation of that distance changed from 9 to 309. So a rule like "look 3 tokens back" would not have a consistent representation.

Sinusoidal Encoding and Relative Shifts

For one sine/cosine pair:

$$v(pos) = [\sin(pos/s), \cos(pos/s)].$$

For a shifted position $pos + k$:

$$v(pos + k) = [\sin((pos + k)/s), \cos((pos + k)/s)].$$

Using:

$$\sin(a+b) = \sin(a)\cos(b) + \cos(a)\sin(b),$$
$$\cos(a+b) = \cos(a)\cos(b) - \sin(a)\sin(b),$$

with $a = pos/s$ and $b = k/s$, we get:

$$\sin((pos+k)/s) = \sin(pos/s)\cos(k/s) + \cos(pos/s)\sin(k/s),$$
$$\cos((pos+k)/s) = \cos(pos/s)\cos(k/s) - \sin(pos/s)\sin(k/s).$$

The important part is that $\cos(k/s)$ and $\sin(k/s)$ depend on k, not on pos.

So the same offset $+3$ has the same transformation structure whether we move from 0 to 3, from 50 to 53, or from 1000 to 1003.

Rotation Interpretation

Moving by k positions acts like a rotation in the sine/cosine plane. The rotation angle depends on k/s.

This is why sinusoidal PE is absolute in construction but relative-friendly in behavior. Each absolute position has its own vector, but relative shifts have predictable mathematical structure.

Chapter 6 Geometric Spacing: The Most Important Intuition

A natural question is:

> If the distance between position 50 and 53 is already 3, why do we need geometric spacing? Shouldn't the model already know that?

The answer is: the model is not directly handed the number 3. It is handed vectors. The Transformer does not automatically subtract raw position numbers like a human does. It receives $PE(50)$ and $PE(53)$, and then the learned attention layers must infer useful relationships from those vectors.

So geometric spacing is not about redefining distance. It is about giving the model many useful signals from which distance can be learned.

One Frequency Is Not Enough

Suppose we used only one fast frequency:

$$PE(pos) = [\sin(pos), \cos(pos)].$$

This changes quickly. It can distinguish nearby positions well. But sine and cosine are periodic, so the pattern repeats. Over long sequences, faraway positions can become ambiguous.

Now suppose we used only one slow frequency:

$$PE(pos) = [\sin(pos/10000), \cos(pos/10000)].$$

This changes very slowly. It is useful for broad or global position, but nearby positions look almost the same.

Therefore:

```
fast frequency -> good for local differences, bad alone for long range
slow frequency -> good for broad position, bad alone for tiny differences
```

The model needs both.

The Ruler Analogy

Geometric spacing gives the Transformer many rulers.

A tiny ruler is good for measuring small differences. A large ruler is good for measuring large distances. You would not measure a city only with a centimeter ruler, and you would not measure a pencil only with a kilometer ruler.

In positional encoding:

```
small scale -> sensitive to nearby positions
large scale -> sensitive to broad position over long sequences
```

So geometric spacing gives many "zoom levels" of position.

What Geometric Spacing Actually Means

The scale for pair i is:

$$s_i = 10000^{2i/d_{model}}.$$

The frequency is:

$$f_i = \frac{1}{s_i} = \frac{1}{10000^{2i/d_{model}}}.$$

As i increases, the scale gets larger and the frequency gets smaller. The scale does not grow like:

```
1, 2, 3, 4, 5, ...
```

It grows multiplicatively, more like:

```
1, 10, 100, 1000, ...
```

The real values are smoother when d_{model} is large, but the concept is the same: the model gets a wide range of scales.

Why Not Linear Spacing?

If we used linearly spaced scales, many scales would be wasted in a narrow range. Geometric spacing covers multiple orders of magnitude efficiently.

Language needs multiple scales:

- local word order: "not good" versus "good not",
- phrase-level structure: "the book on the table",
- sentence-level structure: subject–verb relationships,
- long-range structure: a token near the beginning affecting something much later.

A single scale cannot handle all of these equally well. Geometric spacing gives the model small, medium, and large positional measuring tools.

A Simple Mental Model

Think of each sine/cosine pair as a clock ticking at a different speed.

```
fast clock      -> changes a lot between nearby positions
medium clock    -> changes at a moderate rate
slow clock      -> changes slowly across the sequence
very slow clock -> preserves broad position information
```

The position is represented by all these clock readings together. One clock is ambiguous because it repeats. Many clocks together create a much richer fingerprint.

This is the heart of geometric spacing:

> The model is not given one positional signal. It is given many positional signals at many scales.

Chapter 7 Requirements of Positional Encoding

There is no single official universal list of exactly four requirements, but the common requirements are usually the following.

Requirement 1: Same Dimension as Token Embeddings

The positional encoding must have the same dimension as the token embedding because the original Transformer adds them:

$$X_{input} = X_{token} + PE.$$

If $d_{model} = 512$, both vectors must be 512-dimensional.

Requirement 2: Distinct Position Information

Different positions should produce different patterns. A single sine wave repeats, but many sine/cosine waves at different frequencies produce a rich fingerprint.

Requirement 3: Relative Position Should Be Learnable

The model often needs to learn relationships like previous token, next token, two tokens before, or a distant dependency. Sinusoidal PE helps because a shift by k has a predictable transformation depending on k, not directly on absolute pos.

Requirement 4: Generalization to Longer Sequences

Sinusoidal PE is generated by a formula, so we can compute $PE(600)$ even if the model was trained mainly on shorter sequences.

Requirement 5: Bounded Values

Since sine and cosine are always between -1 and 1, position values stay numerically stable.

Requirement 6: Smoothness

Nearby positions have similar but not identical encodings. This is better than one-hot vectors, where nearby positions are completely unrelated.

Requirement 7: Efficiency

For sequence length n and dimension d_{model}, the PE matrix has shape:

$$[n, d_{model}].$$

It can be precomputed and reused.

Chapter 8 Generalizing to Longer Sequences

Why This Matters

With sinusoidal PE, if the model reaches position 600, we can compute:

$$PE(600) \in \mathbb{R}^{512}.$$

This was one of the questions: if we can create a 512-dimensional vector for position 600, then what was the method that could not do this?

The main comparison is **learned absolute positional embeddings**.

Learned Absolute Positional Embedding Tables

A learned positional embedding table looks like:

```
position 0   -> learned vector
position 1   -> learned vector
position 2   -> learned vector
...
position 511 -> learned vector
```

If the maximum length is 512 and $d_{model} = 512$, the table shape is:

$$[512, 512].$$

Now suppose we want to process a sequence of length 700. The model needs vectors for positions 512 through 699, but the table only goes to 511.

Could We Create a Bigger Table?

Yes, but if the model was trained only on positions 0 through 511, the additional positions may be untrained or poorly trained. They exist as parameters, but the model has not learned how to use them well.

Sinusoidal PE avoids this specific problem because positions are produced by a formula, not looked up from a finite learned table.

Important nuance: sinusoidal PE supports longer positions at the encoding level, but it does not guarantee perfect long-context performance. The rest of the Transformer may still have learned mostly short-context behavior.

Chapter 9 The Positional Encoding Matrix

Shape of the Matrix

For sequence length n and model dimension d_{model}:

$$PE \in \mathbb{R}^{n \times d_{model}}.$$

For example:

```
sequence length = 5
d_model = 8
```

Then:

$$PE \in \mathbb{R}^{5 \times 8}.$$

This means five rows, one row for each position, and eight columns, one value for each embedding dimension.

Rows

Conceptually:

```
PE = [
  PE(position 0),
  PE(position 1),
  PE(position 2),
  PE(position 3),
  PE(position 4)
]
```

Each row is one 8-dimensional vector.

How Many Frequencies?

With $d_{model} = 8$, each position gets 8 PE values, but not 8 frequencies. Since dimensions are paired:

```
dimensions 0 and 1 -> frequency 0
dimensions 2 and 3 -> frequency 1
dimensions 4 and 5 -> frequency 2
dimensions 6 and 7 -> frequency 3
```

So:

```
number of PE values per position = 8
number of sine/cosine frequencies = 4
```

In general, for even d_{model}:

$$\text{number of frequencies} = \frac{d_{model}}{2}.$$

Adding to Token Embeddings

If the token embedding matrix has shape $[5, 8]$ and the PE matrix has shape $[5, 8]$, then:

$$X_{input} = X_{token} + PE$$

also has shape $[5, 8]$.

Chapter 10 How Positional Encoding Enters Self-Attention

After adding positional encoding:

$$Z = X + PE.$$

The Transformer computes:

$$\begin{aligned} Q &= ZW_Q, \\ K &= ZW_K, \\ V &= ZW_V. \end{aligned}$$

Attention is:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V.$$

Because Z contains token meaning and position, the attention scores can depend on both semantic similarity and positional relationship.

This means positional encoding affects what each token searches for, what each token offers to others, and what information gets passed forward.

Chapter 11 Why Addition Instead of Concatenation?

The original Transformer adds positional encodings instead of concatenating them.

If token embeddings and positional encodings are both 512-dimensional:

$$[512] + [512] = [512].$$

Concatenation would produce:

$$[512] \| [512] = [1024],$$

which would change the model dimension and require architectural changes.

Addition keeps the same dimension and lets the learned projection matrices decide how to use semantic and positional information.

Chapter 12 Encoder, Decoder, and Masking

In the encoder, positional encoding is added to the source sequence embeddings. In the decoder, positional encoding is added to the target sequence embeddings.

Causal masking in the decoder is different from positional encoding. Masking says:

> Do not look at future tokens.

Positional encoding says:

> This token is at this position, that token is at that position, and their relationship can be learned.

So masking controls visibility, while positional encoding provides order information.

Chapter 13 Permutation Equivariance: Why Order Is Not Automatic

A common confusion is that the input matrix already has ordered rows:

```
row 0 = first token
row 1 = second token
row 2 = third token
```

But self-attention without positional encoding is permutation-equivariant. If we shuffle the input rows, the output rows are shuffled in the same way. The model does not inherently know that row 0 means first.

For example:

```
dog bites man
man bites dog
```

The words are the same, but the meanings differ because order differs. Positional encoding gives the model the missing order signal.

Chapter 14 Comparison With Other Position Methods

One-Hot Position Encoding

One-hot encoding gives each position a separate vector:

```
position 0 -> [1,0,0,0,...]
position 1 -> [0,1,0,0,...]
position 2 -> [0,0,1,0,...]
```

It gives distinct positions, but nearby positions are completely unrelated.

Binary Position Encoding

Binary encoding can make nearby positions look very different:

```
position 7 -> 00111
position 8 -> 01000
```

A tiny position change may flip many bits.

Learned Absolute Position Embeddings

Learned position embeddings are flexible and often work well, but they rely on a finite learned table. This makes length extrapolation less natural.

Modern Alternatives

Later models often use other methods such as learned position embeddings, relative position bias, RoPE, ALiBi, and other variants. Sinusoidal PE remains important because it explains the original Transformer design and the core position problem very clearly.

Chapter 15 Step-by-Step Flow Through a Transformer

Suppose the input is:

```
I love cats
```

First, tokens are embedded:

```
I    -> E_I
love -> E_love
cats -> E_cats
```

Then positional encodings are added:

$$
\begin{aligned}
Z_0 &= E_I + PE_0, \\
Z_1 &= E_{love} + PE_1, \\
Z_2 &= E_{cats} + PE_2.
\end{aligned}
$$

Then:

$$
\begin{aligned}
Q_i &= Z_i W_Q, \\
K_i &= Z_i W_K, \\
V_i &= Z_i W_V.
\end{aligned}
$$

For token i, attention compares Q_i with all keys K_j. Because Q_i and K_j were computed from position-aware vectors, the attention score can depend on both token meaning and token order.

Chapter 16 Common Misunderstandings

Misunderstanding 1: The Model Is Directly Given Distances

The model is not directly given "distance = 3." It receives vectors. The positional encoding gives those vectors structure so the model can learn relative relationships.

Misunderstanding 2: i Is the Dimension Number

i is the sine/cosine pair index. The raw dimensions are $2i$ and $2i + 1$.

Misunderstanding 3: $d_{model} = 8$ Means 8 Frequencies

With $d_{model} = 8$, each position has 8 PE values, but only 4 sine/cosine frequency pairs.

Misunderstanding 4: Sinusoidal PE Guarantees Perfect Long-Context Performance

It only guarantees that the PE vector can be computed for longer positions. It does not guarantee that the whole model will work perfectly on much longer sequences.

Misunderstanding 5: Positional Encoding Directly Teaches Grammar

PE does not directly encode grammar. It provides order information. The model learns how to use that information during training.

Chapter 17 Final Summary

Sinusoidal positional encoding solves the problem that self-attention does not naturally know token order.

The formula is:

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right),$$

$$PE(pos, 2i+1) = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right).$$

The key ideas are:

- each position gets a vector of size d_{model};
- dimensions are grouped into sine/cosine pairs;
- each pair uses a different frequency;
- frequencies are geometrically spaced;
- small scales capture local differences;
- large scales capture broad or global position;
- sine/cosine values are bounded;
- nearby positions change smoothly;
- relative shifts have predictable structure;
- the formula can produce encodings for unseen longer positions.

The deepest intuition is:

> Sinusoidal positional encoding gives each token many clocks ticking at different speeds. Together, these clocks tell the model where the token is and help it learn how far tokens are from each other.

So each Transformer input vector becomes:

$$\text{meaning} + \text{position}.$$

Then self-attention can compare not only what tokens mean, but also where they are and how they are positioned relative to each other.